

CAPSTONE ASSIGNMENT 2: SELENIUM PYTHON FRAMEWORK DEVELOPMENT (UNITTEST + PYTEST + POM)

OBJECTIVE :

Design and develop a robust Selenium Python Automation Framework using:

- Unittest
- PyTest
- Page Object Model (POM)
- Utility Classes
- Configuration Management
- Test Data Handling (CSV)
- Screenshots on Failure
- HTML Reporting

PROJECT SCENARIO

Automate the Login and Product Search functionality of an E-Commerce application using a scalable framework.

Suggested Application:

- <https://tutorialsninja.com/demo/>
- <https://automationexercise.com/>

Tools, Software, and Concepts Used

- **Programming Language:** Python
- **Automation Tool:** Selenium WebDriver
- **Testing Frameworks:** Unittest, PyTest
- **Design Pattern:** Page Object Model (POM)
- **Browser:** Google Chrome
- **Driver Management:** WebDriver Manager
- **Test Data:** CSV files
- **Configuration:** Configuration file (config.ini)
- **Reporting:** PyTest HTML Report
- **Utilities:** Screenshot capture, explicit waits, reusable utility functions
- **Concepts:** Automation Framework, POM, Test Data Handling, Configuration Management, Assertions, Exception Handling, Modular Testing

SOURCE CODE / IMPLEMENTATION

The framework was implemented using a modular folder structure:

SeleniumFramework/

|

├─ config/

| └─ config.ini

|

├─ testdata/

| └─ login_data.csv

|

├─ pages/

| └─ login_page.py

| └─ search_page.py

|

├─ utilities/

| └─ csv_reader.py

| └─ screenshot.py

|

├─ tests/

| └─ test_login.py

| └─ test_search.py

|

└─ conftest.py

└─ pytest.ini

Source Code :

create_test_user.py

import argparse

from pages.register_page import RegisterPage

from utils.csv_reader import get_test_data

from utils.driver_factory import DriverFactory

def main():

 parser = argparse.ArgumentParser()

```

parser.add_argument("--headless", action="store_true")

args = parser.parse_args()

user = next(row for row in get_test_data("login_data") if row["expected_result"] == "success")

driver = DriverFactory.get_driver(headless=True if args.headless else None)

try:

    page = RegisterPage(driver).open()

    page.register("Capstone", "Tester", user["email"], "9999999999", user["password"])

    if page.is_registration_successful():

        print(f"Account created for {user['email']}")

    else:

        print(f"Registration did not succeed: {page.get_error_message()}")

        print("If the e-mail is already registered, either it is yours already or pick a new "

              "unique e-mail in testdata/login_data.csv and run this script again.")

finally:

    driver.quit()

if __name__ == "__main__":

    main()

```

Requirements.txt

selenium>=4.15.0

pytest>=7.4.0

pytest-html>=4.1.0

Run_unittest.py

```

import argparse

import os

import sys

import unittest

from datetime import datetime

from utils.config_reader import ROOT_DIR, ConfigReader

from utils.html_reporter import HtmlTestResult, write_html_report

def main():

```

```

parser = argparse.ArgumentParser(description="Run unittest suite with HTML report")

parser.add_argument("--browser", help="chrome | firefox | edge")

parser.add_argument("--headless", action="store_true")

parser.add_argument("--pattern", default="test_*.py")

args = parser.parse_args()

# ConfigReader gives environment variables priority over config.ini

if args.browser:

    os.environ["BROWSER"] = args.browser

if args.headless:

    os.environ["HEADLESS"] = "true"

start_dir = ROOT_DIR / "tests" / "unittest_tests"

suite = unittest.defaultTestLoader.discover(

    str(start_dir), pattern=args.pattern, top_level_dir=str(ROOT_DIR))

result = unittest.TextTestRunner(verbosity=2, resultclass=HtmlTestResult).run(suite)

report = ConfigReader.get_path("reports") / f"unittest_report_{datetime.now():%Y%m%d_%H%M%S}.html"

write_html_report(result.records, report, "Selenium Capstone - Unittest Report")

print(f"\nHTML report: {report}")

return 0 if result.wasSuccessful() else 1

if __name__ == "__main__":

    sys.exit(main())

```

test_login_unittest.py :

```

from pages.account_page import AccountPage

from pages.home_page import HomePage

from tests.unittest_tests.base_test import BaseTest

from utils.csv_reader import get_test_data

LOGIN_DATA = get_test_data("login_data")

VALID_USER = next(row for row in LOGIN_DATA if row["expected_result"] == "success")

class LoginUnittest(BaseTest):

    def test_login_page_loads_from_home_page(self):

        login_page = HomePage(self.driver).go_to_login_page()

```

```

        self.assertTrue(login_page.is_loaded(), "Login form was not displayed")

        self.assertIn("Account Login", self.driver.title)

    def test_logout_after_successful_login(self):

        login_page = HomePage(self.driver).go_to_login_page()

        login_page.login(VALID_USER["email"], VALID_USER["password"])

        account_page = AccountPage(self.driver)

        self.assertTrue(account_page.is_loaded())

        account_page.logout()

        self.assertTrue(account_page.is_logout_successful())

    def _build_login_test(row):

        def test(self):

            login_page = HomePage(self.driver).go_to_login_page()

            login_page.login(row["email"], row["password"])

            if row["expected_result"] == "success":

                self.assertTrue(AccountPage(self.driver).is_loaded(),

                                f"Valid login failed for {row['email']}. Run create_test_user.py first.")

            else:

                self.assertIn(row["expected_message"], login_page.get_error_message())

        return test

# One real test method per CSV row, so each row gets its own browser, result and screenshot.
for _row in LOGIN_DATA:

    setattr(LoginUnittest, f"test_{_row['test_id'].lower()}_data_driven", _build_login_test(_row))

test_search_pytest.py :

import pytest

from pages.home_page import HomePage

from utils.csv_reader import get_test_data

SEARCH_DATA = get_test_data("search_data")

@pytest.mark.regression

@pytest.mark.parametrize("row", SEARCH_DATA, ids=[row["test_id"] for row in SEARCH_DATA])

def test_search_data_driven(driver, row):

```

```

results = HomePage(driver).search_for(row["search_term"])

if row["expected_result"] == "found":

    names = results.get_product_names()

    assert len(names) >= int(row["min_count"]), f"Expected >= {row['min_count']} products, got {names}"

    assert any(row["expected_product"].lower() in name.lower() for name in names), (

        f"{row['expected_product']}' not found in {names}"

    )

else:

    assert results.has_no_results(), "Expected the 'no product' message"

    assert "no product that matches" in results.get_no_result_message()

@pytest.mark.smoke

def test_search_results_heading_contains_search_term(driver):

    results = HomePage(driver).search_for("iphone")

    assert "iphone" in results.get_heading().lower()

```

account_page.py :

```

from selenium.webdriver.common.by import By

from pages.base_page import BasePage

class AccountPage(BasePage):

    HEADING = (By.XPATH, "//div[@id='content']/h2[normalize-space()='My Account']")

    # The right sidebar is an <aside id="column-right">, so the locator must not assume a <div>

    LOGOUT_LINK = (By.XPATH, "//*[@id='column-right']/a[normalize-space()='Logout']")

    LOGOUT_HEADING = (By.XPATH, "//div[@id='content']/h1[normalize-space()='Account Logout']")

    def is_loaded(self):

        return self.is_displayed(self.HEADING)

    def logout(self):

        self.click(self.LOGOUT_LINK)

    def is_logout_successful(self):

        return self.is_displayed(self.LOGOUT_HEADING)

```

home_page.py :

```

from selenium.webdriver.common.by import By

```

```

from pages.base_page import BasePage

from pages.login_page import LoginPage

from pages.search_results_page import SearchResultsPage

class HomePage(BasePage):

    SEARCH_INPUT = (By.CSS_SELECTOR, "#search input[name='search']")

    SEARCH_BUTTON = (By.CSS_SELECTOR, "#search button")

    MY_ACCOUNT_MENU = (By.XPATH, "//a[@title='My Account']")

    LOGIN_LINK = (By.XPATH, "//ul[contains(@class,'dropdown-menu')]/a[normalize-space()='Login']")

    def open(self):

        self.open_path("")

        return self

    def search_for(self, term):

        self.type_text(self.SEARCH_INPUT, term)

        self.click(self.SEARCH_BUTTON)

        return SearchResultsPage(self.driver)

    def go_to_login_page(self):

        self.click(self.MY_ACCOUNT_MENU)

        self.click(self.LOGIN_LINK)

        return LoginPage(self.driver)

```

login_page.py :

```

from selenium.webdriver.common.by import By

from pages.base_page import BasePage

class LoginPage(BasePage):

    PATH = "index.php?route=account/login"

    RETURNING_CUSTOMER_HEADING = (By.XPATH, "//h2[normalize-space()='Returning Customer']")

    EMAIL_INPUT = (By.ID, "input-email")

    PASSWORD_INPUT = (By.ID, "input-password")

    LOGIN_BUTTON = (By.CSS_SELECTOR, "input[value='Login']")

    ERROR_ALERT = (By.CSS_SELECTOR, "div.alert-danger")

    def open(self):

```

```

        self.open_path(self.PATH)

        return self

    def is_loaded(self):

        return self.is_displayed(self.RETURNING_CUSTOMER_HEADING) and
self.is_displayed(self.LOGIN_BUTTON)

    def login(self, email, password):

        self.type_text(self.EMAIL_INPUT, email)

        self.type_text(self.PASSWORD_INPUT, password)

        self.click(self.LOGIN_BUTTON)

    def get_error_message(self):

        return self.get_text(self.ERROR_ALERT)

```

register_page.py :

```

from selenium.webdriver.common.by import By

from pages.base_page import BasePage

class RegisterPage(BasePage):

    """Used only by create_test_user.py to prepare the account for the valid-login test."""

    PATH = "index.php?route=account/register"

    FIRST_NAME = (By.ID, "input-firstname")

    LAST_NAME = (By.ID, "input-lastname")

    EMAIL = (By.ID, "input-email")

    TELEPHONE = (By.ID, "input-telephone")

    PASSWORD = (By.ID, "input-password")

    CONFIRM = (By.ID, "input-confirm")

    AGREE = (By.NAME, "agree")

    CONTINUE = (By.CSS_SELECTOR, "input[value='Continue']")

    SUCCESS_HEADING = (By.XPATH, "//div[@id='content']/h1[contains(., 'Your Account Has Been Created')]")

    ERROR_ALERT = (By.CSS_SELECTOR, "div.alert-danger")

    def open(self):

        self.open_path(self.PATH)

        return self

    def register(self, first_name, last_name, email, telephone, password):

```

```

self.type_text(self.FIRST_NAME, first_name)

self.type_text(self.LAST_NAME, last_name)

self.type_text(self.EMAIL, email)

self.type_text(self.TELEPHONE, telephone)

self.type_text(self.PASSWORD, password)

self.type_text(self.CONFIRM, password)

self.click(self.AGREE)

self.click(self.CONTINUE)

def is_registration_successful(self):

    return self.is_displayed(self.SUCCESS_HEADING)

def get_error_message(self):

    return self.get_text(self.ERROR_ALERT)

```

search_results_page.py :

```

from selenium.webdriver.common.by import By

from selenium.webdriver.support import expected_conditions as EC

from pages.base_page import BasePage

class SearchResultsPage(BasePage):

    HEADING = (By.XPATH, "//div[@id='content']/h1[starts-with(normalize-space(),'Search')]")

    PRODUCT_NAMES = (By.CSS_SELECTOR, "#content .product-layout .caption h4 a")

    NO_RESULT_MESSAGE = (

        By.XPATH, "//div[@id='content']/p[contains(normalize-space(),'no product that matches')]")

    def wait_for_results(self):

        self.wait.until(

            EC.any_of(

                EC.presence_of_element_located(self.PRODUCT_NAMES),

                EC.presence_of_element_located(self.NO_RESULT_MESSAGE),))

    def get_heading(self):

        return self.get_text(self.HEADING)

    def get_product_names(self):

        self.wait_for_results()

```

```
        return [element.text.strip() for element in self.driver.find_elements(*self.PRODUCT_NAMES)]

def has_no_results(self):

    self.wait_for_results()

    return bool(self.driver.find_elements(*self.NO_RESULT_MESSAGE))

def get_no_result_message(self):

    return self.get_text(self.NO_RESULT_MESSAGE)
```

I also use 2 test_datasets 1. Test_login.csv 2. Test_search.csv

Output with clearly labelled screenshots :

Selenium Capstone - Unittest Report

Generated: 2026-09-21 00:20:25

Total: 12Passed: 12Failed: 0Errors: 0Skipped: 0

#	Test	Status	Time	Details
1	tests.unittest_tests.test_login_unittest.LoginUnit.test_login_page_loads_from_home_page	PASS	5.55s	
2	tests.unittest_tests.test_login_unittest.LoginUnit.test_logout_after_successful_login	PASS	7.54s	
3	tests.unittest_tests.test_login_unittest.LoginUnit.test_tc_login_01_data_driven	PASS	6.69s	
4	tests.unittest_tests.test_login_unittest.LoginUnit.test_tc_login_02_data_driven	PASS	6.14s	
5	tests.unittest_tests.test_login_unittest.LoginUnit.test_tc_login_03_data_driven	PASS	6.30s	
6	tests.unittest_tests.test_login_unittest.LoginUnit.test_tc_login_04_data_driven	PASS	6.86s	
7	tests.unittest_tests.test_search_unittest.SearchUnit.test_search_results_heading_contains_search_term	PASS	5.08s	
8	tests.unittest_tests.test_search_unittest.SearchUnit.test_tc_search_01_data_driven	PASS	5.94s	
9	tests.unittest_tests.test_search_unittest.SearchUnit.test_tc_search_02_data_driven	PASS	5.52s	
10	tests.unittest_tests.test_search_unittest.SearchUnit.test_tc_search_03_data_driven	PASS	5.19s	
11	tests.unittest_tests.test_search_unittest.SearchUnit.test_tc_search_04_data_driven	PASS	4.99s	
12	tests.unittest_tests.test_search_unittest.SearchUnit.test_tc_search_05_data_driven	PASS	5.14s	

Selenium Capstone - PyTest Report

Report generated on 21-Sep-2026 at 00:46:47 by [pytest-html](#) v4.2.0

Environment

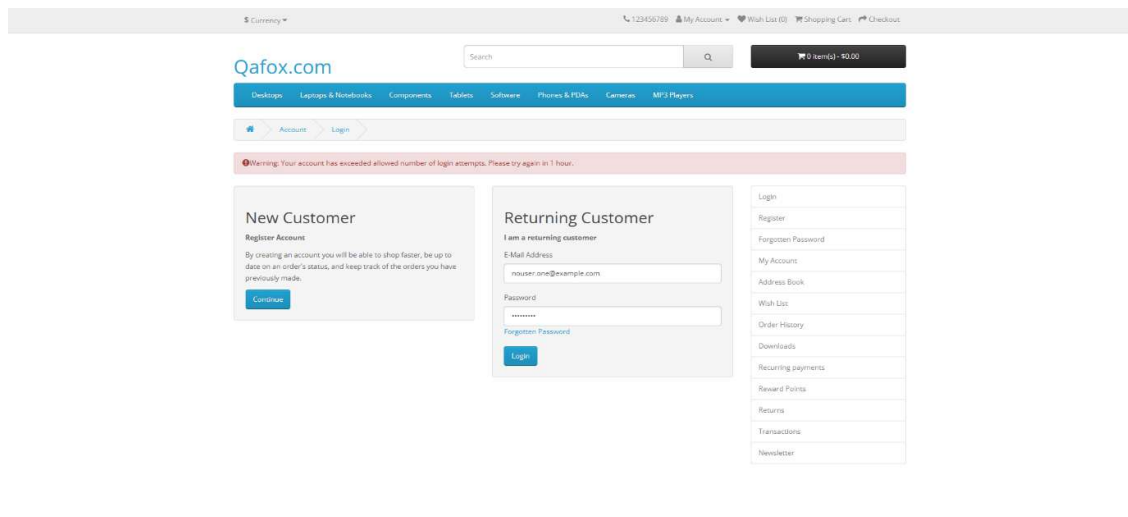
Python	3.11.9
Platform	Windows-10-10.0.26200-SP0
Packages	• pytest: 9.1.1 • pluggy: 1.6.0
Plugins	• html: 4.2.0 • metadata: 3.1.1

Summary

12 tests took 00:01:09.

(Un)check the boxes to filter the results.

Result	Test	Time	Details
Passed	tests\pytest_tests\test_login_pytest.py::test_login_page_loads_from_home_page	00:00:05	
Passed	tests\pytest_tests\test_login_pytest.py::test_login_data_driven[TC_LOGIN_01]	00:00:07	
Passed	tests\pytest_tests\test_login_pytest.py::test_login_data_driven[TC_LOGIN_04]	00:00:07	
Passed	tests\pytest_tests\test_login_pytest.py::test_logout_after_successful_login	00:00:07	
Passed	tests\pytest_tests\test_search_pytest.py::test_search_data_driven[TC_SEARCH_01]	00:00:05	
Passed	tests\pytest_tests\test_search_pytest.py::test_search_data_driven[TC_SEARCH_02]	00:00:05	
Passed	tests\pytest_tests\test_search_pytest.py::test_search_data_driven[TC_SEARCH_03]	00:00:05	
Passed	tests\pytest_tests\test_search_pytest.py::test_search_data_driven[TC_SEARCH_04]	00:00:05	
Passed	tests\pytest_tests\test_search_pytest.py::test_search_data_driven[TC_SEARCH_05]	00:00:05	
Passed	tests\pytest_tests\test_search_pytest.py::test_search_results_heading_contains_search_term	00:00:06	



RESULT:

The Selenium Python framework successfully automated the **Login** and **Product Search** functionalities of the selected e-commerce application.

The framework was able to:

- Launch the browser.
- Open the application.
- Read test data from CSV.
- Perform login operations.
- Search for products.
- Validate expected results.
- Capture screenshots when tests failed.
- Generate an HTML test execution report.

OBSERVATION:

- **POM** made the framework more organized and reusable.
- **PyTest** simplified test execution and reporting.
- **Unittest** provided a standard testing structure.
- CSV-based test data allowed multiple test cases to be executed without changing the test code.
- Configuration management made application settings easier to maintain.
- Screenshots helped identify the cause of failed test cases.
- HTML reports provided a clear summary of test execution.

CONCLUSION:

The project successfully demonstrated the development of a **scalable Selenium Python automation framework** using **Unittest, PyTest, Page Object Model, utility classes, configuration management, CSV test data, screenshots, and HTML reporting**. The framework improves code reusability, maintainability, and the efficiency of automated testing for e-commerce applications.

Name: Aasish Shrestha , Enrollment No: 12023002001003 , CSE , Roll No: 33

Institute of Engineering & Management, Kolkata

Email ID: aasish.shrestha2023@iem.edu.in
aasishshrestha2005@gmail.com